

Requirements Analysis & Use Case Extraction Project

Project Overview

This Claude project is designed to serve as an intelligent requirements analysis assistant, specializing in the extraction and structuring of use cases and user stories from requirements documents and functional analysis specifications. Built on 20+ years of business analysis best practices, this project automates one of the most time-consuming and critical tasks in software development and project management: translating business requirements into actionable, structured specifications.

The system combines natural language processing capabilities with domain expertise in requirements engineering to provide business analysts, project managers, and development teams with professionally formatted, comprehensive documentation. Whether you're working with detailed functional specifications or high-level business requirements, this project intelligently identifies actors, extracts use cases, infers user stories, and organizes everything into a clear, role-based structure.

This automated approach not only saves significant time but also ensures consistency, completeness, and traceability across your requirements documentation, making it an invaluable tool for agile teams, traditional waterfall projects, and hybrid methodologies alike.

Key Features

Intelligent Document Analysis

- **Automatic Actor Identification:** Recognizes all users, actors, and stakeholders mentioned or implied in requirements documents
- **Use Case Extraction:** Identifies explicitly stated use cases and intelligently infers them from functional requirements
- **User Story Generation:** Creates well-formed user stories following industry best practices
- **Dependency Tracking:** Maps relationships and dependencies between different use cases and user stories

Structured Output Generation

- **Role-Based Organization:** Groups use cases and user stories by user role for clear ownership and responsibility
- **Standardized Format:** Each use case includes Code, Name, Title, Target, Input Data, Output Data, and Dependencies
- **Consistent Numbering:** Automatic sequential numbering with role-specific prefixes (e.g., ADM-001, USR-003)
- **Relevance-Weighted Detail:** More critical features receive more detailed analysis and documentation

Multi-Format Export Options

- **Word Documents (.docx):** Professional formatted documents ready for stakeholder review
- **Excel Spreadsheets (.xlsx):** Structured tables perfect for tracking, filtering, and analysis
- **Markdown Files (.md):** Developer-friendly format ideal for version control and collaboration
- **Multiple Formats:** Generate several formats simultaneously for different audiences

Multi-Language Support

- **Language Selection:** Choose your preferred output language before generation
- **English Default:** English language offered as default option for international teams
- **Localized Output:** All generated content in the selected language

Quality Assurance Features

- **Ambiguity Detection:** Flags unclear or contradictory requirements
- **Completeness Checking:** Ensures all use cases have required information
- **Consistency Validation:** Verifies consistent formatting and structure throughout
- **Evidence-Based Analysis:** All extractions based on actual document content, not assumptions

Output Structure

The system generates comprehensive documentation organized by user role. Each role section contains all associated use cases and user stories with the following standardized structure:

Use Case Format

[Code]: [IdentifierPrefix]-[SequentialNumber]

- Example: (ADM-001), (USR-012), (DEV-005)

[Name]: Descriptive name

- Example: (User Registration), (Generate Monthly Report), (Approve Purchase Order)

[Title]: [IdentifierPrefix]-[SequentialNumber]: descriptive name

- Example: (ADM-001: Configure System Settings)

[Target]: Brief description of the main goal

- Example: (Enable administrators to configure global system parameters and preferences to customize the application behavior according to organizational requirements)

[Input Data]: List of all required and optional inputs

- **Mandatory:**

- Authentication credentials
- Configuration parameters
- Validation rules
- **Optional:**
 - Custom templates
 - Logo and branding assets
 - Email notification preferences

[Output Data]: List of all outputs, results, and effects

- Updated system configuration
- Configuration change log entry
- Confirmation notification to administrator
- System restart trigger (if required)
- Audit trail record

Dependencies: Related use cases

- Example: USR-001: User Authentication, ADM-003: View Configuration History

Workflow Process

The project follows a streamlined, user-friendly workflow designed to minimize manual effort while maintaining full control over the output:

Step 1: Document Upload

User uploads their requirements document or functional analysis specification in supported formats (PDF, DOCX, TXT, MD).

Step 2: Language Selection

System prompts: "Which language should be used for the output document? (Default: English)"

- User selects preferred language
- Or accepts English default by pressing Enter

Step 3: Format Selection

System prompts: "Please choose the output format(s):"

1. Word Document (.docx)
2. Excel Spreadsheet (.xlsx)

3. Markdown File (.md)
4. Multiple formats (specify which)

Step 4: Automated Analysis

The system performs comprehensive analysis:

- Reads and understands document content
- Identifies all actors and roles
- Extracts or infers use cases and user stories
- Assigns role identifiers and sequential numbers
- Extracts targets, inputs, and outputs
- Maps dependencies
- Assesses relevance levels

Step 5: Output Generation

System generates requested format(s) as downloadable artifacts with:

- Complete role-based organization
- Standardized formatting
- Professional presentation
- Clear file naming

Step 6: Download

User receives downloadable artifact(s) ready for immediate use in their project.

Use Cases

Use Case 1: Agile Sprint Planning

Scenario: A Scrum Master receives a high-level product vision document from stakeholders and needs to break it down into specific user stories for sprint planning.

Application: Upload the vision document, select Markdown format for easy integration with Jira/GitHub, and receive structured user stories organized by persona (End User, Administrator, API Consumer) with clear acceptance criteria derived from inputs and outputs.

Benefit: Transforms a 20-page vision document into sprint-ready user stories in minutes, enabling the team to begin estimation and planning immediately.

Use Case 2: Stakeholder Communication

Scenario: A Business Analyst needs to present functional requirements to non-technical stakeholders who

prefer traditional documentation formats.

Application: Upload technical functional specifications, select Word format, and generate a professionally formatted document with clear use case descriptions organized by business role (Customer, Manager, Operator).

Benefit: Bridges the gap between technical specifications and business understanding, facilitating stakeholder approval and reducing miscommunication.

Use Case 3: Quality Assurance Test Planning

Scenario: A QA Lead needs to create a test plan based on functional requirements but the requirements are scattered across multiple documents and lack clear structure.

Application: Upload consolidated requirements, select Excel format, and receive a structured spreadsheet with use cases, inputs, and expected outputs that can be directly mapped to test cases.

Benefit: Provides complete traceability between requirements and test cases, ensuring comprehensive test coverage and easier defect tracking.

Use Case 4: Multi-Language Project Documentation

Scenario: An international project team needs requirements documentation in multiple languages for distributed development teams.

Application: Upload English requirements, generate multiple outputs in different languages (English for offshore team, Italian for local team), maintaining consistent structure across all versions.

Benefit: Ensures all teams work from identical functional specifications in their preferred language, reducing translation errors and misunderstandings.

Technical Requirements

Supported Input Formats

- **PDF:** Text-extractable PDF documents
- **Word Documents:** .docx, .doc formats
- **Text Files:** .txt format
- **Markdown:** .md format
- **Rich Text:** .rtf format

Supported Output Formats

- **Word Documents:** .docx (Microsoft Word 2010+)
- **Excel Spreadsheets:** .xlsx (Microsoft Excel 2010+)
- **Markdown:** .md (CommonMark standard)

Input Document Characteristics

- **Language:** Any language (output can be translated)
- **Length:** From 1 page to 500+ pages
- **Structure:** Structured, semi-structured, or unstructured
- **Content Types:** Requirements, specifications, user needs, business processes, functional descriptions

System Capabilities

- **Natural Language Understanding:** Advanced comprehension of requirements terminology and business contexts
- **Inference Engine:** Intelligent derivation of implicit use cases from functional descriptions
- **Consistency Checking:** Automated validation of logical consistency and completeness
- **Dependency Analysis:** Graph-based relationship mapping between use cases

Limitations

- **Highly Technical Code:** May require manual review for low-level implementation details
- **Ambiguous Requirements:** Flags unclear sections but cannot resolve contradictions without user input
- **Non-Standard Terminology:** Works best with industry-standard terminology; custom terminology should be defined in the document
- **Visual Diagrams:** Extracts information from text; complex UML diagrams or flowcharts may need manual interpretation

Benefits and Value Proposition

Time Savings

- **80% Reduction** in manual requirements documentation time
- **Instant Structure** applied to unstructured requirements
- **Batch Processing** of multiple documents in sequence

Quality Improvement

- **Consistent Formatting** across all project documentation
- **Complete Coverage** ensuring no use cases are missed
- **Standardized Structure** making review and approval faster

Team Collaboration

- **Multiple Formats** for different team preferences (developers prefer Markdown, managers prefer Word)
- **Clear Organization** by role improving accountability

- **Dependency Mapping** highlighting integration points

Compliance and Traceability

- **Audit Trail** through structured documentation
- **Requirement IDs** for traceability to design and testing
- **Version Control** friendly formats (Markdown)

Getting Started

1. **Create Project:** Set up a new Claude Project using the project description
2. **Add Instructions:** Copy the project instructions into the Project Instructions field
3. **Upload Document:** Drag and drop your requirements document
4. **Select Options:** Choose language and output format(s)
5. **Generate:** Receive your structured use case documentation
6. **Refine:** Iterate if needed by providing feedback or uploading additional context

Best Practices

For Best Results

- **Include Context:** Provide background information about the project domain
- **Define Terms:** Include a glossary if using specialized terminology
- **Structure Helps:** Even basic structure (headings, sections) improves extraction accuracy
- **Iterate:** Start with a section, review results, then process the full document

Document Preparation Tips

- **Clear Role Definitions:** Explicitly mention user types and actors
- **Action Verbs:** Use clear action verbs for system behaviors (create, update, delete, notify)
- **Input/Output Clarity:** Specify what data goes in and what comes out
- **Include Examples:** Real examples help the system understand context

Support and Feedback

For optimal results, this project works best when:

- Requirements documents are reasonably clear and complete
- User roles are mentioned or can be inferred from context
- Business processes are described with sufficient detail
- The analyst provides feedback to refine output

The system continuously improves its extraction accuracy through iterative use within the same project context, learning from corrections and clarifications provided during the conversation.